

ATLANTIC COMPUTATIONAL EXCELLENCE NETWORK

Introduction to Shell Scripting

Dr. Ross Dickson, Computational Research Consultant

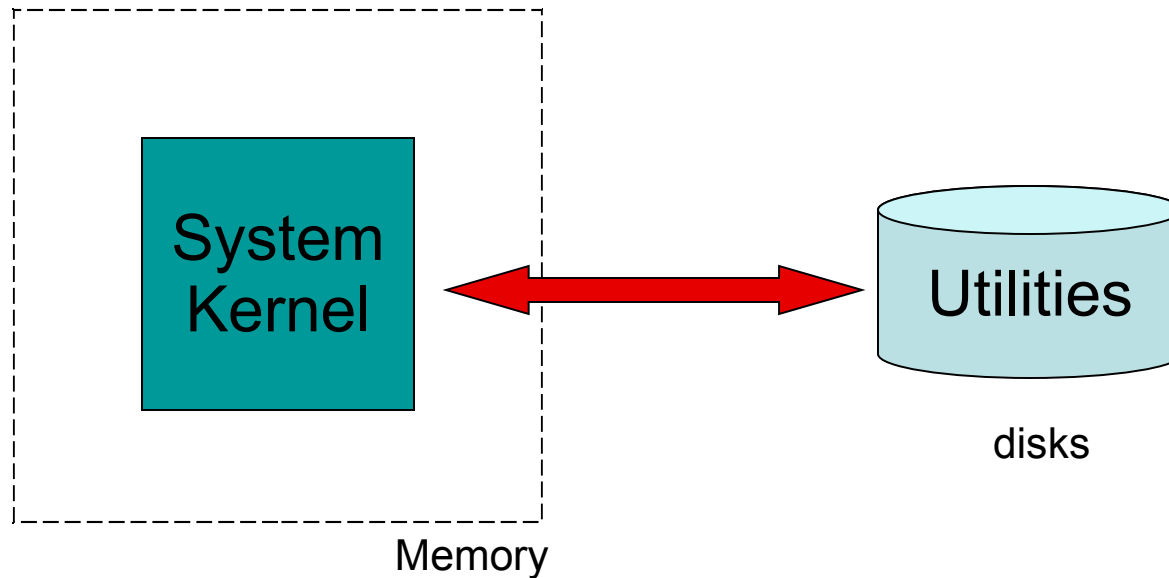
What is the Shell?

It is...

- ...a program
- ...a programming language
- ...how you interact with the operating system

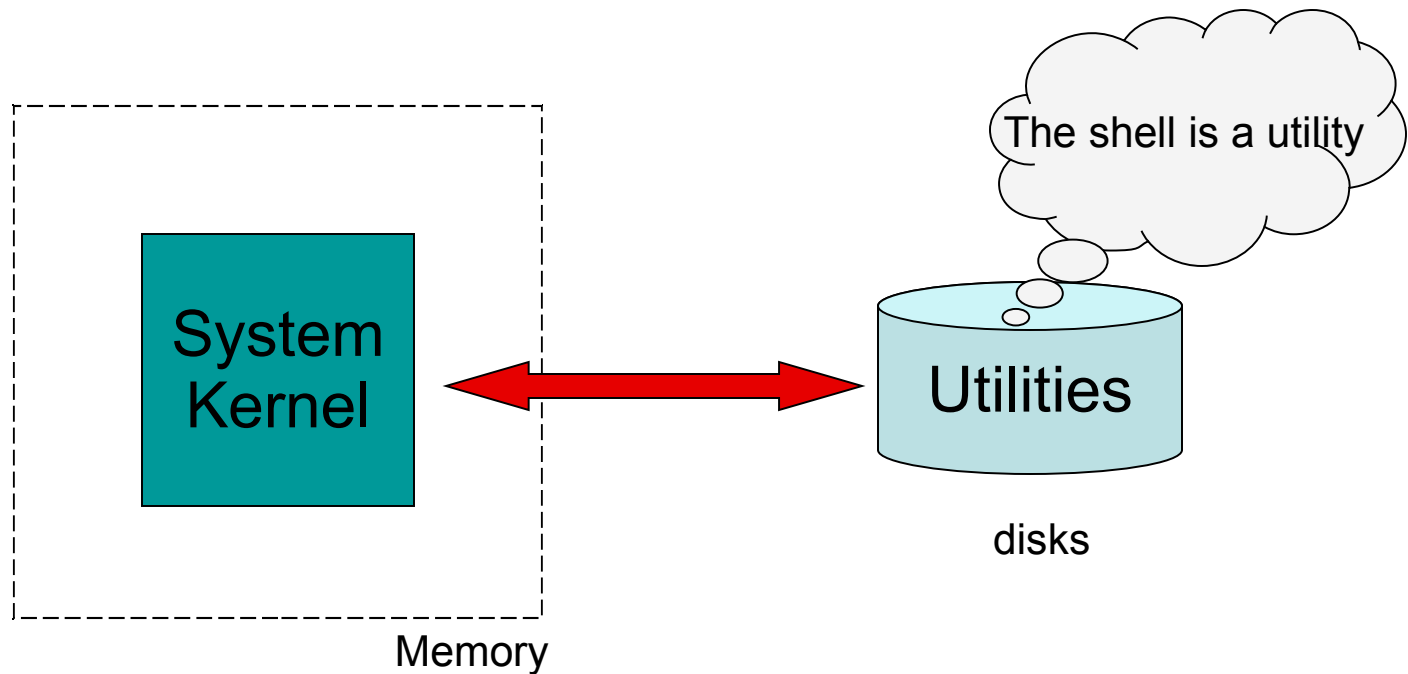
What is the Shell?

The Unix/Linux system is divided into the *kernel* and *utilities*



What is the Shell?

The Unix/Linux system is divided into the *kernel* and *utilities*



A first shell script

first.sh

```
echo "It is now" `date`  
echo  
who  
echo  
echo "The number of login windows is" $(who | wc -l)  
echo  
echo "My user name is" $(whoami) "and I am in  
directory" $(pwd)
```

The output is:

```
rdickson@glooscap: ~/demo > chmod +x first.sh
```

```
rdickson@glooscap: ~/demo > ./first.sh
```

```
It is now Thu Oct 30 10:54:20 ADT 2008
```

```
skhan      pts/0      Oct 30 08:29 (140.184.24.201)
```

```
skhan      pts/3      Oct 30 08:47 (140.184.24.201)
```

```
vchevrie   pts/5      Oct 30 09:42 (happy.phys.dal.ca)
```

```
rdickson   pts/6      Oct 30 10:41 (rdickson.ucis.dal.ca)
```

```
The number of login windows is 4
```

```
My user name is rdickson and I am in directory
```

```
/home/rdickson/demo
```

```
rdickson@glooscap: ~/demo >
```

A practical example:

Suppose you have
fifty jobs on the cluster,
some in r state,
some in qw state,
and some in Eqw state.

How do you delete your Eqw jobs efficiently?

qstat output

```
$ qstat
```

job-ID	prior	name	user	state	submit/start	at	queue
4320603	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:22:58	long.q
4320606	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:23:14	long.q
4320607	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:23:14	long.q
4320604	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:22:58	long.q
4320605	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:22:58	long.q
4320600	0.50000	qgenNettsZ	thauser	r	05/21/2010	15:22:28	long.q
4319822	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:17	
4319823	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:22	
4319826	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:30	
4319828	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:35	
4319829	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:41	
4319831	0.50000	qpostPro	thauser	qw	05/21/2010	14:13:47	
4319837	0.50000	qpostPro	thauser	Eqw	05/21/2010	14:14:05	
4319839	0.50000	qpostPro	thauser	Eqw	05/21/2010	14:14:11	
4319841	0.50000	qpostPro	thauser	Eqw	05/21/2010	14:14:17	
4319843	0.50000	qpostPro	thauser	Eqw	05/21/2010	14:14:23	

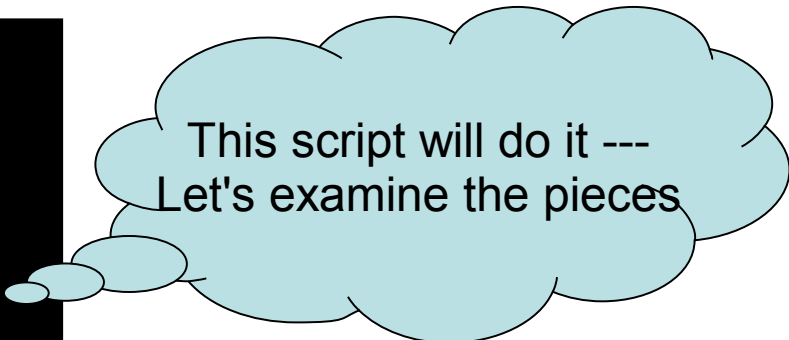
We need to 'qdel 4319837', 'qdel 4319839',

The answer

Suppose you have fifty jobs on the cluster, some in *r* state, some in *qw* state and some in *Eqw* state

How do you delete your *Eqw* jobs efficiently?

```
qstat > temp1
grep 'Eqw' temp1 > temp2
cut -c1-8 temp2 > temp3
sed 's/^/qdel /' temp3 > del.sh
source del.sh
rm temp1 temp2 temp3 del.sh
```

A light blue thought bubble with a black outline, containing text. It has three smaller circles leading to it from the left.

This script will do it ---
Let's examine the pieces

Eqw.sh

Finding lines: grep



```
grep 'Jac' phonebook
```

```
Jacob 902 555 1234
```

```
grep -v 'Jac' phonebook
```

```
James 902 555 4567
Jane 902 555 2365
Keith 902 555 5673
Kenny 902 555 0989
Larry 902 555 6575
```

phonebook			
Jacob	902	555	1234
James	902	555	4567
Jane	902	555	2365
Keith	902	555	5673
Kenny	902	555	0989
Larry	902	555	6575

PATTERNS

a.c

a[xyz]c or a[x-z]c

a[^xyz]c

ab*c

ab+c

^abc

abc\$

a(bc|de)f

MATCHED STRINGS

a[any single character]c, e.g. abc, a1c, a c.

axc, ayc, and azc only

a[any single character but x, y, or z]c

a[0 or more b]c, e.g. ac, abc, abbbc

a[1 or more b]c, e.g. abc, abbbc

abc only at the beginning of a line

abc only at the end of a line

abcf and adef

Extracting columns: cut

```
cut -c1-5 phonebook
```

```
Jacob  
James  
Jane  
Keith  
Kenny  
Larry
```

```
cut -c1-6,16- phonebook
```

```
Jacob 1234  
James 4567  
Jane 2365  
Keith 5673  
Kenny 0989  
Larry 6575
```

If you want space-separated fields, try 'awk' instead.

Extracting columns: cut

```
a#b#c#d#e#f#g#h#i#j#12#14#16 test_cut
```

```
cut -c1 test_cut
```

```
a
```

```
cut -c1-5 phonebook
```

```
a#b#c
```

```
cut -c1-5,15 test_cut
```

```
a#b#ch
```

```
cut -d# -f1 test_cut
```

What do
-d and -f do?

Re-assembling them: paste



test_paste1

```
aaa
bbb
ccc
dddd
eeee
ffff
```

test_paste2

```
902 555 1234
902 555 4567
902 555 2365
902 555 5673
902 555 0989
902 555 6575
```

`paste test_paste1 test_paste2`

```
aaa 902 555 1234
bbb 902 555 4567
ccc 902 555 2365
dddd 902 555 5673
eeee 902 555 0989
ffff 902 555 6575
```

`paste -d ':' test_paste1 test_paste2`

```
aaa:902 555 1234
bbb:902 555 4567
ccc:902 555 2365
dddd:902 555 5673
eeee:902 555 0989
ffff:902 555 6575
```

`paste -s test_paste1`

```
aaa bbb ccc dddd eeee ffff
```

Stream editor sed



sed 's/902/ /' phonebook

```
Jacob      555 1234
James      555 4567
Jane       555 2365
Keith      555 5673
Kenny      555 0989
Larry      555 6575
```

sed '4,5d' phonebook

```
Jacob      902 555 1234
James      902 555 4567
Jane       902 555 2365
Larry      902 555 6575
```

sed 's/^/hello /' phonebook

```
hello Jacob 902 555 1234
hello James 902 555 4567
hello Jane  902 555 2365
hello Keith 902 555 5673
hello Kenny 902 555 0989
hello Larry 902 555 6575
```

Stream editor sed



test_paste2

sed 's/902/ /' test_paste2

```
555 1234
555 4567
555 2365
555 5673
555 0989
555 6575
```

```
902 555 1234
902 555 4567
902 555 2365
902 555 5673
902 555 0989
902 555 6575
```

sed '5d' test_paste2

```
902 555 1234
902 555 4567
902 555 2365
902 555 5673
902 555 6575
```

sed 's/^/hello /' test_paste2

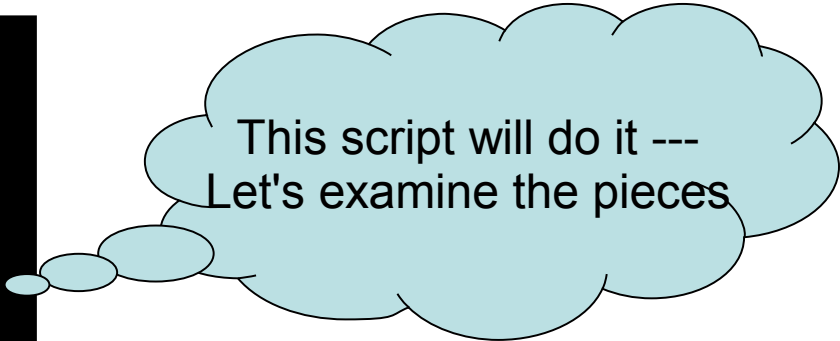
```
hello 902 555 1234
hello 902 555 4567
hello 902 555 2365
hello 902 555 5673
hello 902 555 0989
hello 902 555 6575
```

Back to the example...

Suppose you have fifty jobs on the cluster, some in *r* state, some in *qw* state and some in *Eqw* state

How do you delete your *Eqw* jobs efficiently?

```
qstat > temp1
grep 'Eqw' temp1 > temp2
cut -c1-8 temp2 > temp3
sed 's/^/qdel /' temp3 > del.sh
source del.sh
rm temp1 temp2 temp3 del.sh
```

A light blue thought bubble with a black outline, containing text. It has three smaller circles leading to the main bubble from the left.

This script will do it ---
Let's examine the pieces

Eqw.sh

Use the pipe!

Avoid all those annoying intermediate files with “the pipe”:



pipeEqw.sh

```
qstat | grep Eqw | cut -c1-8 | sed 's/^/qdel /' > del.sh  
source del.sh  
rm del.sh
```

Programming: if – then – else

If
then
else
fi

```
#!/bin/sh
user="$1"
if who | grep "^$user " > /dev/null
then
    echo "$user is logged ON"
else
    echo "$user is not logged on"
fi
```

iftest.sh

- \$1, \$2, *etc.* are script arguments
- \$user is a variable
 - set with “name=”
 - referenced with “\$name”

Programming: if – then – else

If
then
else
fi

```
#!/bin/sh
user="$1"
if who | grep "^$user " > /dev/null
then
    echo "$user is logged ON"
else
    echo "$user is not logged on"
fi
```

iftest.sh

- Dialect-dependent!
 - /bin/tcsh has different syntax
 - first line `#!` tells system which shell to use

File testing example

```
#!/bin/sh

if [ -f $1 ]
then
    echo "Yes, $1 is here"
else
    echo "No file named $1 here"
fi
```

iftest2.sh

File tests

Operator	Returns TRUE (exit status of 0) if
<code>-d file</code>	<i>file is a directory</i>
<code>-e file</code>	<i>file exists</i>
<code>-f file</code>	<i>file is an ordinary file</i>
<code>-r file</code>	<i>file is readable by the process</i>
<code>-s file</code>	<i>file has nonzero length</i>
<code>-w file</code>	<i>file is writable by the process</i>
<code>-x file</code>	<i>file is executable</i>
<code>-L file</code>	<i>file is a symbolic link</i>

Another practical example

You want to submit a number of jobs, differing only in the Temperature in the input file and you want each to have a unique job name.

Another practical example



We'll want these files...

run.exe

template.in

sub.sh

```
#####  
#Temperature  
T=1  
#Pressure  
P=1  
#
```

```
#$ -cwd  
#$ -N CHANGE  
#$ -o t.out  
#$ -e t.err  
#$ -l h_rt=12:00:00  
  
./run.exe input"$1".in
```

Programming: 'for' loop



make_input.sh

```
#!/bin/sh
# Make input files for a range of temperatures
j=0
for t in 178 189 208 213 218 228 248
do
    j=$((j+1))
    sed 's/T=1/T='$t'/'      template.in > input"$j".in
    sed 's/CHANGE/job'$j'/' sub.sh      > sub"$j".sh
    echo temperature=$t : qsub sub"$j".sh $j
done
```

Look into “here documents” for a way to do this without 'sed'.
Bash syntax shown, tcsh syntax is different.

Where next?

- Online manual, 'man'
 - `man sh` or `man bash` or `man tcsh`
- Tutorials on the Web
 - Google 'unix shell script tutorial'
- Books
 - *The Unix Programming Environment* by Kernighan & Pike is the classic
 - ...but there are many others!
- ACEnet Wiki
 - <http://wiki.ace-net.ca>
- ACEnet Tech Support email
 - `support@ace-net.ca`

Example files

`glooscap.ace-net.ca:/home/rdickson/demo/Scripting/*`